Screen Share App

A production-minded low-latency screen sharing application built with **React**, **Node.js**, **WebRTC**, **mediasoup**, and **Socket.IO**.

The platform supports:

- one-to-many screen sharing
- multiple rooms
- multiple concurrent users
- optional audio sharing
- live ephemeral room messaging
- low-latency media delivery over WebRTC
- scalable SFU-based architecture

This project is designed as a serious MVP, not a toy demo.

Features

Core

- Create room
- Join room as broadcaster or viewer
- Share screen with remote participants
- Multiple viewers in the same room
- Multiple active rooms at the same time
- Viewer count updates
- Broadcaster start/stop controls
- Realtime connection state updates

Media

- Screen sharing using `getDisplayMedia()`
- WebRTC media transport
- SFU architecture using **mediasoup**
- UDP-based delivery through WebRTC where possible
- TURN/STUN support for NAT traversal

Live Messaging

- Room-based live chat
- Ephemeral messages only
- No DB storage
- No replay to late joiners
- Messages vanish on disconnect / session end
- System messages for join, leave, start share, stop share

Reliability / Engineering

- Clean frontend/backend separation
- Environment-based config
- Socket event contract
- Health endpoint
- Graceful cleanup on disconnect
- Docker-based local setup support

Tech Stack

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- Zustand
- React Router
- Socket.IO client
- mediasoup-client
- WebRTC browser APIs

Backend

- Node.js
- Express
- TypeScript

- Socket.IO
- mediasoup
- UUID

Infra

- coturn
- Docker Compose

Architecture

Why this architecture

This application uses an **SFU (Selective Forwarding Unit)** architecture instead of peer-to-peer mesh.

Why not mesh

Mesh does not scale well for one-to-many screen sharing:

- broadcaster uploads separate streams to each viewer
- CPU and bandwidth usage grows fast
- performance degrades quickly as users increase

Why SFU

With SFU:

- broadcaster sends one upstream media stream
- server forwards media to multiple viewers
- lower sender bandwidth
- better scalability for multi-user rooms
- cleaner control over rooms, peers, and stream lifecycle

Why mediasoup

`mediasoup` was chosen because it gives:

- strong control over WebRTC transport lifecycle
- scalable SFU behavior

- production-grade media routing
- Node.js integration
- flexibility for custom room and signaling logic

Application Flow

Broadcaster Flow

- User creates or joins a room as broadcaster
- Frontend connects to backend over Socket.IO
- Frontend requests WebRTC transport creation
- User starts screen sharing via `navigator.mediaDevices.getDisplayMedia()`
- Frontend creates a producer transport
- Screen video track is produced to mediasoup
- Backend marks the broadcaster as active in that room
- Viewers are notified that a producer is available

Viewer Flow

- User joins a room as viewer
- Frontend connects via Socket.IO
- Frontend requests receive transport creation
- When broadcaster is live, server notifies viewers
- Viewer creates a consumer for the producer
- Consumed media track is attached to a MediaStream
- Stream is rendered in a video element

Messaging Flow

- User sends a room message
- Client emits message via Socket.IO
- Server validates payload
- Server broadcasts message to currently connected room participants
- Message is rendered in client chat panel
- Message is not stored or replayed later

Frontend Responsibilities

The React frontend is responsible for:

- Room create/join UI
- Broadcaster and viewer pages
- Local screen capture with `getDisplayMedia()`
- `mediasoup-client` transport handling
- Receiving and rendering remote media
- Connection and status indicators
- Live ephemeral chat UI
- Room lifecycle cleanup on leave/disconnect

Backend Responsibilities

The Node.js backend is responsible for:

- Express server setup
- Socket.IO signaling
- mediasoup worker/router lifecycle
- Room creation and room lookup
- Peer lifecycle management
- Transport creation/connection
- Producer and consumer coordination
- Viewer count updates
- Broadcaster status tracking
- Ephemeral message fanout
- Cleanup when users disconnect

Room Model

Each Room Maintains

- Room ID
- mediasoup router
- Peer collection
- Active broadcaster state
- Current producer(s)

- Transient state required for active session handling

Each Peer Maintains

- Socket ID
- Display name
- Role (broadcaster or viewer)
- Send/recv transports
- Producers
- Consumers

WebSocket / Socket.IO Details

Socket.IO is used for:

- Signaling
- Room coordination
- Ephemeral live messaging

It is not used for media transport. Media always flows over WebRTC, not over WebSockets.

Client -> Server Events

- `createRoom`
- `joinRoom`
- `createWebRtcTransport`
- `connectTransport`
- `startScreenShare`
- `consume`
- `resumeConsumer`
- `stopScreenShare`
- `leaveRoom`
- `sendRoomMessage`

Server -> Client Events

- `roomCreated`
- `joinedRoom`
- `transportCreated`

- screenShareStarted
- newProducer
- consumed
- viewerCountUpdated
- broadcasterStopped
- roomMessage
- systemMessage
- errorMessage

High-Level Architecture

```
+-------------------+         Socket.IO Signaling         +-----------------
|   React Client    |  <--------------------------------> |   Node.js Server
|                   |                                      |  Express + Socke
| - Home / Join UI  |                                      | - Room Manager
| - Broadcaster UI  |                                      | - Peer Manager
| - Viewer UI       |                                      | - mediasoup Work
| - Chat Panel      |                                      | - mediasoup Rout
| - mediasoup-client|                                      | - Transport Hand
+---------+---------+                                      +----------+------
          |                                                            |
          |                 WebRTC Media Transport                      |
          +-------------------------------------------------------------+
                                 via mediasoup SFU
                                          |
                                          v
                                   +-----------+
                                   |  coturn   |
                                   | STUN/TURN |
                                   +-----------+
```